Submission Worksheet

Submission Data

Course: IT202-008-S2026
Assignment: IT202 Milestone 2
Student: Kurt C. (kc784)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 802.00/802.00 (100.00%)
Received Grade: 0.00/802.00 (0.00%)
Started: 4/25/2026 2:25:48 PM
Updated: 4/26/2026 4:24:40 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-2/grading/kc784
View Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-2/view/kc784

Instructions

- Overview Link: https://youtu.be/t6X8dzZt_a0

1. Refer to Milestone2 of this doc: https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone2 branch
    1. `git checkout Milestone2` (ensure proper starting branch)
    2. `git pull origin Milestone2` (ensure history is up to date)
5. Fill out the below worksheet
    - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    - Ensure proper styling is applied to each page
    - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone2`
    4. On Github merge the pull request from `Milestone2` to `qa`
    5. On Github create a pull request from `qa` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the Render.com prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
    1. `git checkout qa`
    2. `git pull origin qa`

Section #1: (2 pts.) Data

Progress: 100%

≔ Task #1 (0.50 pts) - API Data Table

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the table(s) structure of the table made to hold your API
- There should be at least 3 properties/fields beyond `id`, `created`, `modified`, and respective `is_api` or `api_id` columns
- Logical types and constraints should be set
- Show at least a few records from each API table (samples should include both API fetched data and custom entries noted by an `api_id` or `is_api column)

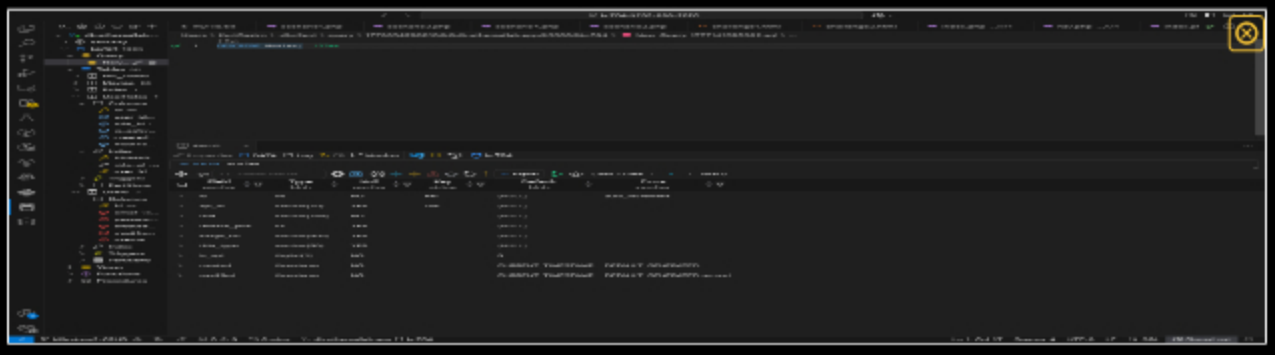
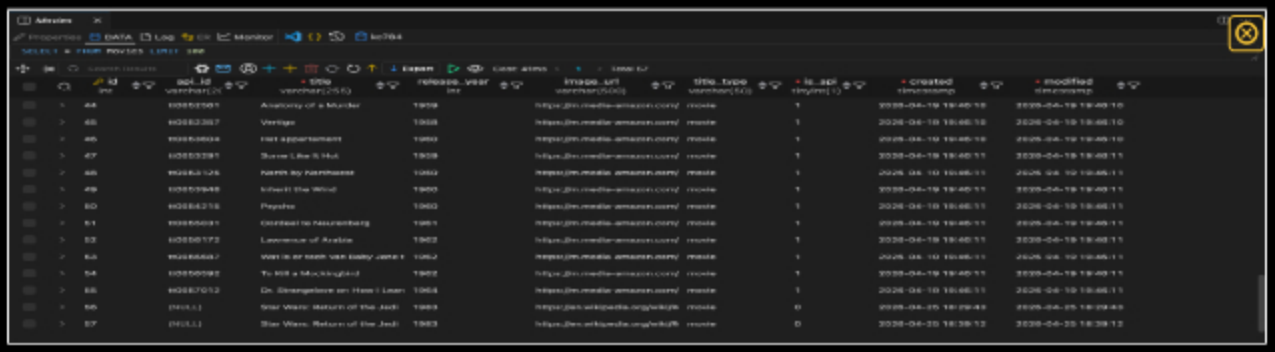


table structure



records with sample movies

Saved: 4/25/2026 2:59:39 PM

Part 2:

Progress: 100%

Details:

- Add a direct link to your sql file from Github (should end in `.sql`)

URL #1
https://github.com/kurtjcastro/kc784-

URL
https://github.com/kurtjcastro/kc

it202-008-2026/public_html/project/sql/006_create_table_movies.sql

Saved: 4/25/2026 2:59:39 PM

Part 3:

Progress: 100%

Details:

- Briefly note what each field/property represents

Your Response:

The id acts as the primary key and unque identifier for each row. The api_id is the ID from the API or NULL for manually added movies. The title is the name of the movie. The release_year is the year the movie came out. The image_url is a URL to the movie poser. The title_type is the category such as movie, short, tvMovie, etc. is_api is either 0 if manually created or 1 if taken from API. created is a timestamp of when a record was added. modified is the timestamp it was last updated.

Saved: 4/25/2026 2:59:39 PM

Task #2 (0.50 pts.) - Data Transformation

Progress: 100%

Part 1:

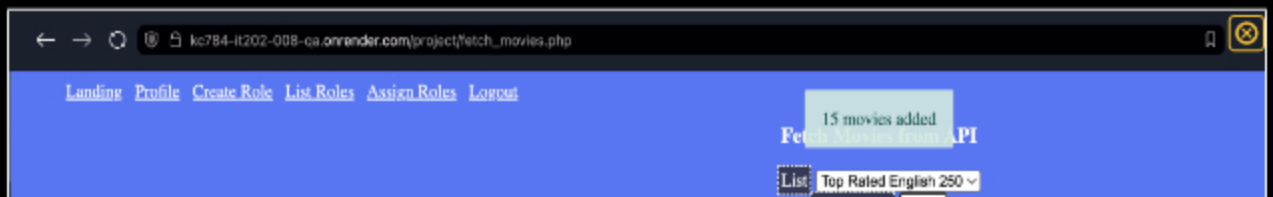
Progress: 100%

Details:

- Show the code that handles the data transformation/conversion (from the API)
- Show a Render.com qa example of API getting fetched and loaded into the DB



relevant code





API getting fetched



list of movies loaded into DB

Saved: 4/25/2026 3:14:53 PM

Part 2:

Progress: 100%

Details:

- Include Render.com prod link to API fetch page
- Include pull request link for this feature

URL #1
https://kc784-it202-008-prod.onrender.com/project/fetch_movies.php

URL
https://kc784-it202-008-prod.onre

URL #2
https://github.com/kurtjcastro/kc784-it202-008-20[illegible]39

URL
https://github.com/kurtjcastro/kc

Saved: 4/25/2026 3:14:53 PM

Part 3:

Progress: 100%

Details:

- Will you be using all the data or just a subset?
- Do you need to convert any data or extract pieces of a property's value?
- Briefly explain the transformation/extraction logic to get the API data to the shape your application intends to use

Your Response:

For this project, I will only be using a subset as the API returns most of the data for each movie

For this project, I will only be using a subset as the API returns most of the data for each movie such as the IDs, image dimesions, caption, typename, original title text, release data, etc. As for this project, I only extracted the fields that were relevant to my application, the ID, title, release year, post URL, and title type.

Yes, the API returned nested objects so I needed to extract the values into the bare variables in order to match my table structure. I didn't need to convert types since the values return as strings and ints already.

The API response has an array of movie objects with their own objects. I looped through each result and only extracted specific properties into variables such as $movie["titleText"]["text"] into $title. This skips any movie missing a tititle or API ID and checks if the movies already exists in my DB with the api_id to avoid any duplicates and inserts new entries with is_api=1 and manual entries with is_api = 0

Saved: 4/25/2026 3:14:53 PM

Task #3 (0.50 pts.) - API Duplicates

Progress: 100%

Details:

- Consider how duplicate entries are handled
  - What happens if you get the same result from the API for an entity that exists but has been unmodified?
  - What happens if you get the same result from the API for an entity that exists but has been modified?

Your Response:

Before adding a movie, a record is checked by looking if it shares the same api_id as another record in the database. If a match is found, the new movie is not added to the database and moves on to the next movie insertion in order to prevent duplicate.

If the entity exists but has been unmodified then it gets skipped since the api_id already exists in the database. The code skips with a continue and doesn't touch any of the existing records.

If the record was modified then the API fetch will skip it because the matching api_id triggers the continue anyway and the local changes are saved to avoid being overwritten by the API. If changes were made on the API side then those changes are ignored since they are skipped instead of updated to protect manual edits.

Saved: 4/25/2026 3:18:22 PM

Task #4 (0.50 pts.) - API Fetching

Progress: 100%

Details:

- Consider how your application will trigger this data fetch
  - (i.e., periodic via cron job, poor man's cron job, on app start/awake, etc)
  - (i.e., User or Admin triggered via a specific page/action)
  - (i.e., Lazy loaded during searches for stale/missing content)

Your Response:

The data fetch is triggered manually only from logged in users through fetch_movies.php. A dropdown pops up with available lists of movies they want to pull data from and how many results they would like to fetch and must click submit to send the request. Each API call is user initiated, which preserves quota usage on any possible background requests.I chose this approach because it was a simpler approach than cron jobs or lazy loading for this particular project.

Saved: 4/25/2026 3:22:12 PM

Section #2: (150 pts.) Data Creation

Progress: 100%

Task #1 (0.60 pts.) - Validations

Progress: 100%

Part 1:

Progress: 100%

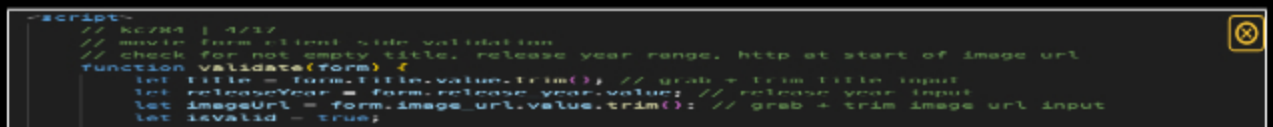
Details:

- Show the html validations (code)
- Show the javascript validations (code)
- Show the php validations (code)

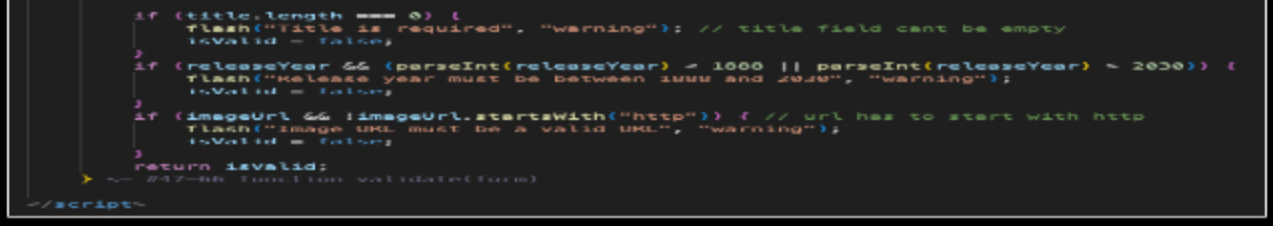


```
<!-- kc284 | 4/17 -->
<h2>Add Movie</h2>  <!-- movie form -->
<form onsubmit="return validate(this)" method="POST"> <!-- 1$ validation before submission -->
    <div>
        <label for="title">Title:</label>
        <!-- for sticky value from post -->
        <input type="text" id="title" name="title" required maxlength="255" value="<?php echo $_POST['title'] ?? '' ?>" />
    </div>
    <div>
        <label for="release_year">Release Year:</label>
        <input type="number" id="release_year" name="release_year" min="1888" max="2050" value="<?php echo($_POST['release_year']); ?>" />
    </div>
    <div>
        <label for="image_url">Image URL:</label>
        <input type="url" id="image_url" name="image_url" maxlength="500" value="<?php echo($_POST['image_url']); ?>" />
    </div>
    <div>
        <label for="title_type">title type:</label>
        <!-- dropdown form selection -->
        <select id="title_type" name="title_type">
            <option value="">-- select --</option>
            <option value="movie" <?php if (($_POST['title_type'] ?? '') === 'movie') echo 'selected'; ?>>Movie</option>
            <option value="tvSeries" <?php if ($_POST['title_type'] === 'tvSeries') echo 'selected'; ?>>TV Series</option>
            <option value="short" <?php if ($_POST['title_type'] === 'short') echo 'selected'; ?>>Short</option>
            <option value="tvMovie" <?php if ($_POST['title_type'] === 'tvMovie') echo 'selected'; ?>>TV Movie</option>
            <option value="tvMiniSeries" <?php if ($_POST['title_type'] === 'tvMiniSeries') echo 'selected'; ?>>TV Mini-Series</option>
        </select>
    </div>
    <input type="submit" value="Add Movie" /> <!-- submit -->
</form>
```

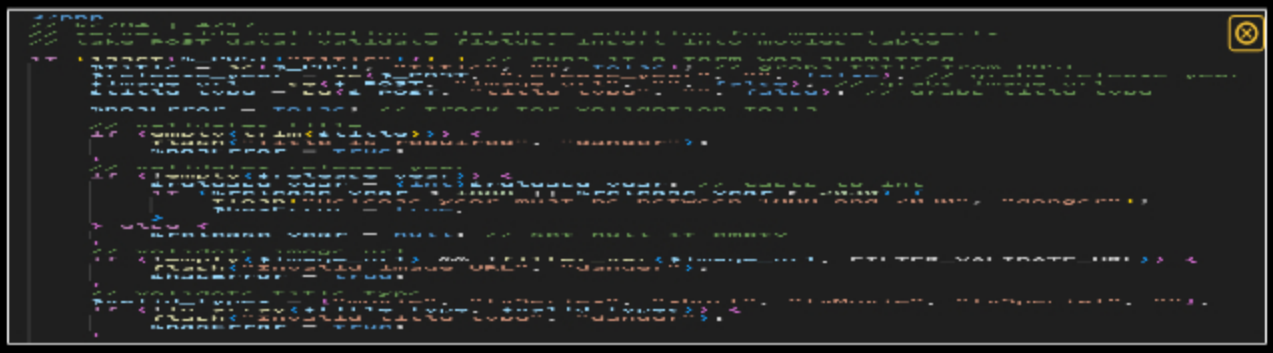
html validation



```
<script>
    // kc284 | 4/17
    // movie form client side validation
    // check for not empty title, release year range, http at start of image url
    function validate(form) {
        let title = form.title.value.trim(); // grab + trim title input
        let releaseYear = form.release_year.value; // release year input
        let imageUrl = form.image_url.value.trim(); // grab + trim image url input
        let invalid = true;
```



js validation



php validation

Saved: 4/25/2026 3:31:55 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the html validations used
- Briefly explain the javascript validation logic
- Briefly explain the php validation logic
- Note which role(s) have access to this create form

Your Response:

The HTML attributes are used for basic rules before form submission where the title field has required so it can't be empty and a max length of 255 chars. Release year uses type="number" with a minimum of 1888 and max of 2030 so only movies within that range are accepted. Image URL uses type="url" to enforce proper URL format with a max length of 500 and title type uses

Saved: 4/25/2026 3:31:55 PM

Task #2 (0.60 pts.) - Examples

Progress: 100%

Details:

- Show a successful creation
- Show how a duplicate submission is handled (i.e., what message or behavior occurs)

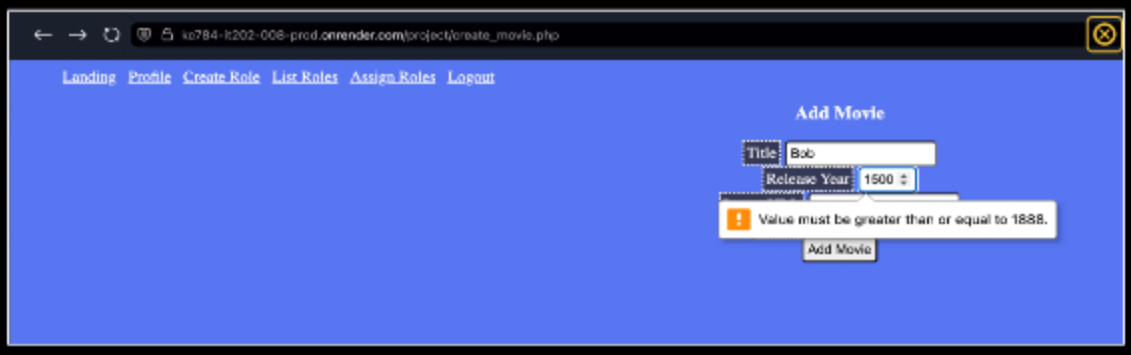
- Demonstrate various validation errors
- Ensure Render.com prod url is visible



successful creation



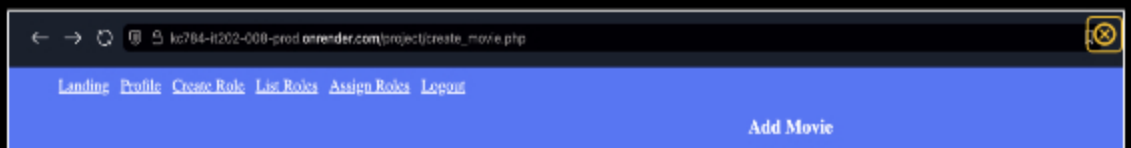
0 movies added if duplicate



invalid release year



invalid url





empty title

Saved: 4/25/2026 3:37:45 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1
https://kc784-it202-008-prod.onrender.com/project/create_movie.php

URL
https://kc784-it202-008-prod.

URL #2
https://github.com/kurtjcastro/kc784-it202-008-2026/pull/39

URL
https://github.com/kurtjcastr

Saved: 4/25/2026 3:39:29 PM

Section #3: (150 pts.) Data List Page (Many Items)

Progress: 100%

Task #1 (0.60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

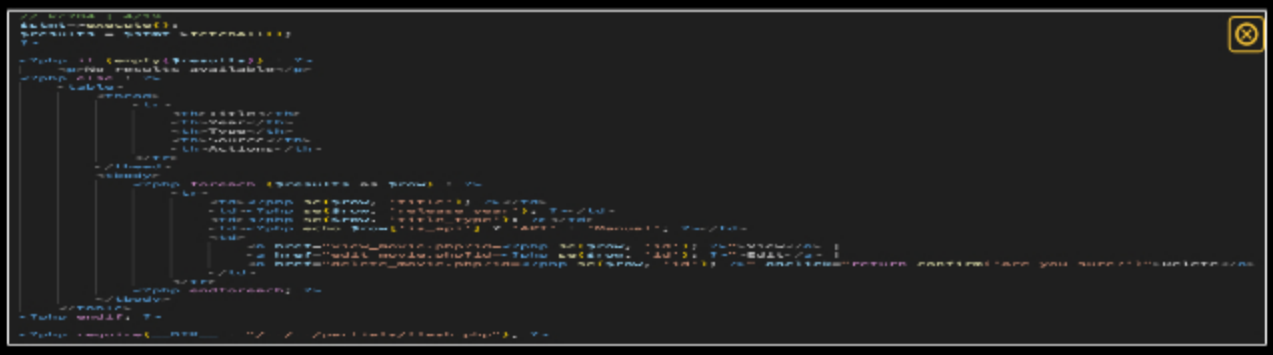
- Show the code that handles the filter/sort logic
- Show the code that enforces the record limit (valid range 1-100, default 10 if out of range)
- Show the code that generates the list/grid output
- Styling must be applied



filter logic

```
// kc784 | 4/19
// ensure limit is in range
if ($limit < 1 || $limit > 100) {
    $limit = 10;
}
```

record limit



list output

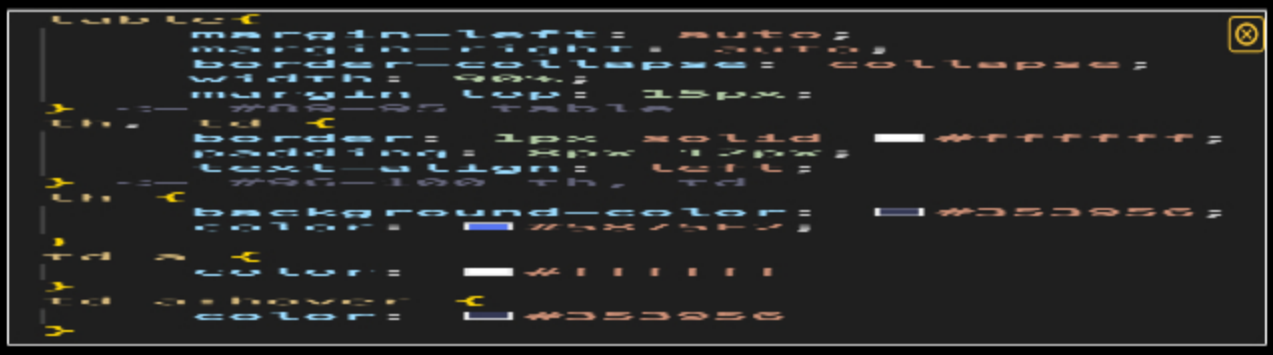


table styling

Saved: 4/26/2026 3:00:35 PM

Part 2:

Progress: 100%

Details:

Briefly explain how the filter/sort logic works and formulates the results

- Briefly explain how the filter/sort logic works and formulates the results
- Briefly explain how the limit is validated and applied server-side
- Briefly explain how the list/grid output is generated
- Briefly note the styling choices

Your Response:

The filter/sort logic works through the page reading the filter values from the URL query with the GET params. By making the SQL query dynamically, a like clause is used for partial matching and if a title type is selected then a match clause is added. The conditions are stored within $filters array and joined with AND and using implode. The sort columnare validated against a whitelist of allowed values to help protect against SQL injection attacks and appended as ORDER BY.

The limit is validated by grabbing the limit value from the GET params and are cast to an integer. If the int falls outside the range of 1-100, it defaults to 10. This makes sure that users aren't able to request zero, negative, or more than 100 results.

The list is generated after the query executes as the results are fetched into an array. If the array is empty, then a "No results available" message is shown, otherwise a foreach loop goes through each row and outputs it as a table with columns for title, year, type, source, and the action links for viewing, editing, and deleting.

The styling choices uses a dark blue header with blue text to match the color scheme of the site and the white borders to contrast the blue background. Table links are white and darken when hovered and the table is centered with 90% width for a simpler and cleaner look.

Saved: 4/26/2026 3:00:35 PM

Task #2 (0.60 pts.) - Examples

Progress: 100%

Details:

- Show a few variations of filter/sort applied (including no records - should display a "no results" message)
- Show both API-fetched and manually created entities present in the list
- Show the limit field in use with at least two different values
- Each list item should be a summary
- Each list item should have the following links
  - A link to a single view of the specific entity (i.e., a details page)
  - A link to delete this entity (this may be an admin-only functionality, but it should be present for the respective role)
  - A link to edit this entity (this may be an admin-only functionality, but it should be present for the respective role)
- Ensure Render.com prod url is visible





default list (manual + API entries)



filtered for star wars



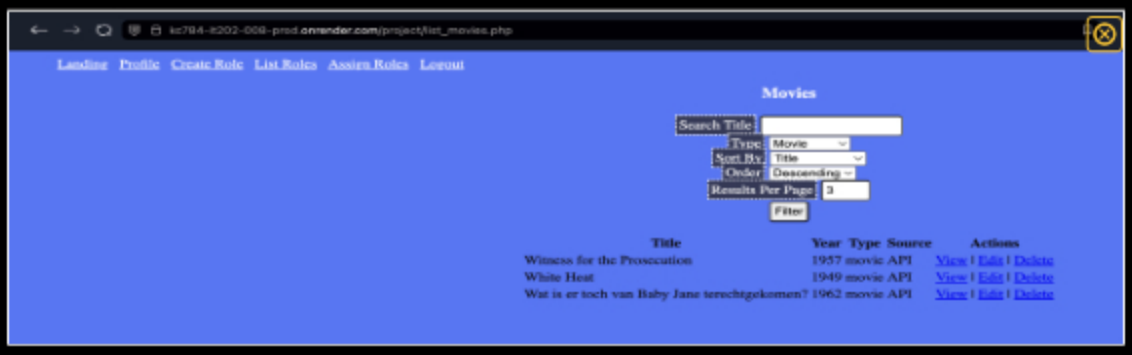
type filter (movie)



descending order



no results



limit to 3 results



limit to 1 result

Saved: 4/26/2026 3:06:16 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1
https://github.com/kurtjcastro/kc784-it202-008-2026/pull/39

URL
https://github.com/kurtjcastr

URL #2
https://kc784-it202-008-prod.onrender.com/project/list_movies.php?title=&title_type=movie&sort=title&order=ASC&limit=10

URL
https://kc784-it202-008-prod.

Saved: 4/26/2026 3:07:49 PM

Section #4: (150 pts.) View Details Page (Single Item)

Progress: 100%

Task #1 (0.60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
  - Include the code that handles invalid/missing ids (should redirect back to the previous page with an appropriate message)
- Show the code that generates the entity output
- Styling must be applied

```php
// kc784 | 4/19
$id = se($_GET, "id", -1, false); // grab id from url or default -1

if ($id < 1) { // check id validity
    flash("Invalid movie id", "danger");
    header("Location: list_movies.php"); // redirect back to list
    die();
} <-- #14-19 if ($id < 1)
$db = getDB();
$stmt = $db->prepare("SELECT * FROM Movies WHERE id = :id"); // grab movie by id
$stmt->execute([":id" => $id]);
$movie = $stmt->fetch(); // grabs single row

if (!$movie) { // if movie not found
    flash("Movie not found", "danger");
    header("Location: list_movies.php");
    die();
} <-- #24-28 if (!$movie)
?>
```

grab id + load record

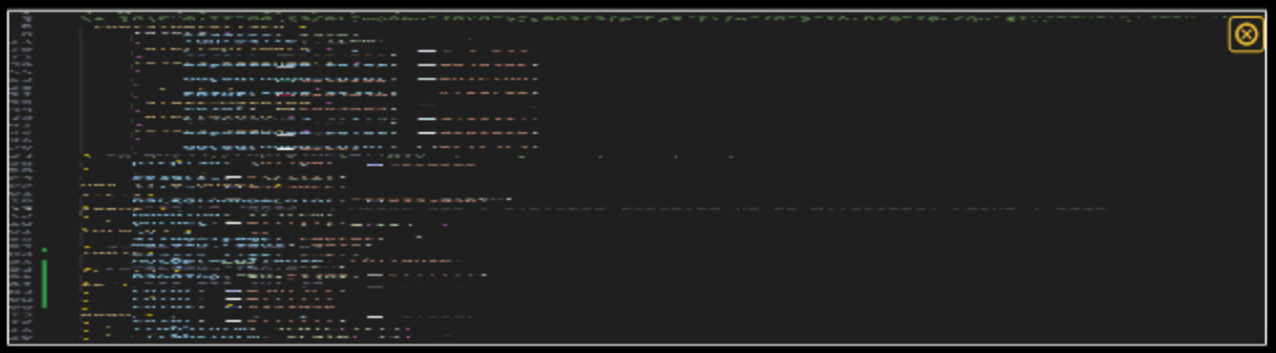
```php
<!-- kc784 | 4/19 -->
<h3><?php se($movie, 'title'); ?></h3> <!-- title as heading -->

<div>
    <p><strong>Release Year:</strong> <?php echo se($movie, 'release_year', 'N/A', false); ?></p>
    <p><strong>Type:</strong> <?php echo se($movie, 'title_type', 'N/A', false); ?></p>
    <p><strong>Source:</strong> <?php echo $movie['is_api'] ? 'API' : 'Manual'; ?></p>
    <p><strong>API ID:</strong> <?php echo se($movie, 'api_id', 'N/A', false); ?></p>
    <p><strong>Added:</strong> <?php se($movie, 'created'); ?></p>
    <p><strong>Last Modified:</strong> <?php se($movie, 'modified'); ?></p>

    <?php if (!empty($movie['image_url'])) : ?> <!-- shows image if available -->
        <p><img src="<?php se($movie, 'image_url'); ?>" alt="<?php se($movie, 'title'); ?>" style="max-width: 200px;" /></p>
    <?php endif; ?>
</div>

<div>
    <!-- links -->
    <a href="edit_movie.php?id=<?php se($movie, 'id'); ?>">Edit</a> |
    <a href="delete_movie.php?id=<?php se($movie, 'id'); ?>" onclick="return confirm('Are you sure?')">Delete</a> |
    <a href="list_movies.php">Back to List</a>
</div>
```

output



styling

Saved: 4/26/2026 3:16:44 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the entity output is generated
- Briefly note the styling choices

Your Response:

The logic works through the page grabbing the movie id from the URL query using $_GET. If the id is less than 1 or is empty, an error flash message is shown and redirects back to the list page. If the id is valid, it queries the db to fetch a movie by that id. If no matching records are found, an error flash essage is shown and redirected again to ensure that both ids are valid.

The entity output is generated by having the title display as the page heading. The property is shown as a labeled paragraph through **and the se() helper to output the value or N/A for empty fields. The source field checks is_api to either display API or Manual depending on 1 or 0 as the value. If the movie has an image URL, the poster is displayed with a max width of 200px and also includes edit, delete, and return to list links.**

The detail page uses the initial site's blue background and white text (i don't know why it keeps defaulting to black text). The poster image is capped at 200px so it doesn't take too much space, and action links are white to match the rest of the site.

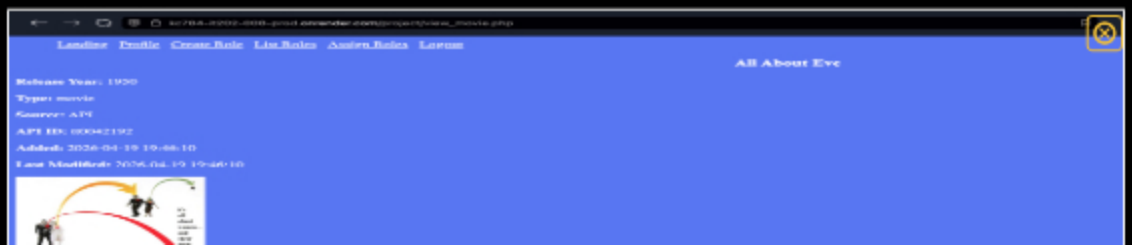
Saved: 4/26/2026 3:16:44 PM

Task #2 (0.60 pts.) - Examples

Progress: 100%

Details:

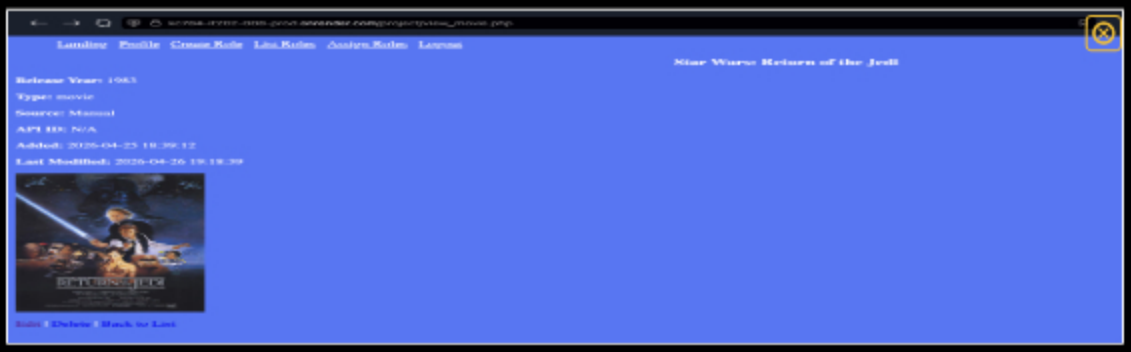
- Show a few examples of different entities
- Each entity item should have more details than the summary view (if possible)
- Each entity should have the following links
  - A link to edit the single entity (this may be an admin-only thing, but it should be present for the respective role)
  - A link to delete the single entity (this may be an admin-only thing, but it should be present for the respective role)



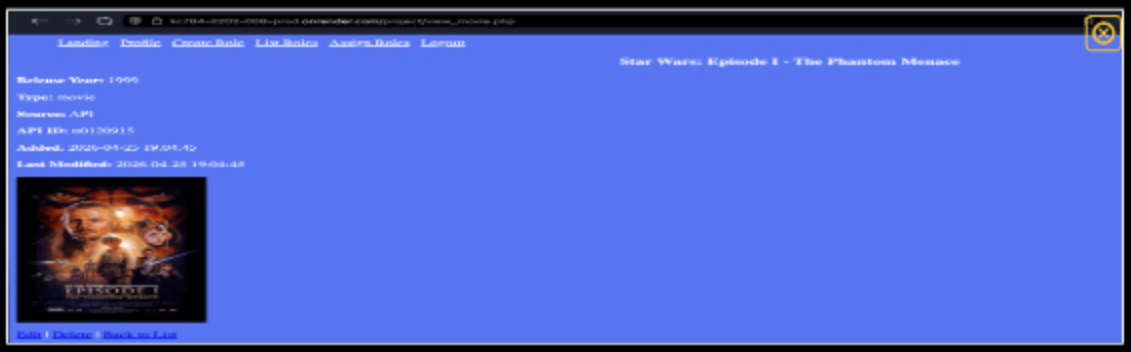


Edit | Delete | Back to List

API fetched movie



manual movie entry



API fetched movie (2)

Saved: 4/26/2026 3:19:49 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1
https://github.com/kurtjcastro/kc784-it202-008-2026/pull/39

URL
https://github.com/kurtjcastr

URL #2
https://kc784-it202-008-prod.onrender.com/project/view_movie.php?id=64

URL
https://kc784-it202-008-prod.

URL #3
https://kc784-it202-008-prod.onrender.com/project/view_movie.php?

URL
https://kc784-it202-008-prod.

id=57

URL #4
https://kc784-it202-008-prod.onrender.com/project/view_movie.php?id=30

URL
https://kc784-it202-008-prod.

Saved: 4/26/2026 3:20:47 PM

Section #5: (150 pts.) Edit Page

Progress: 100%

Task #1 (0.60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
  - Include the code that handles invalid/missing ids
- Show the code that generates the form with the correct data types and populates it
- Show the html, javascript, and php validation (should be similar to create)
- Styling must be applied

```php
// kc784 | 4/19
$id = se($_GET, "id", -1, false); // get id from url

if ($id < 1) { // check if valid id
    flash("Invalid movie id", "danger");
    header("Location: list_movies.php"); // redirect to list
    die();
} <-- #15-19 if ($id < 1)

$db = getDB();
$stmt = $db->prepare("SELECT * FROM Movies WHERE id = :id"); // fetch movie by id
$stmt->execute([":id" => $id]);
$movie = $stmt->fetch(); // grabs single row

if (!$movie) { // if movie not found
    flash("Movie not found", "danger");
    header("Location: list_movies.php");
    die();
} <-- #26-30 if (!$movie)
```

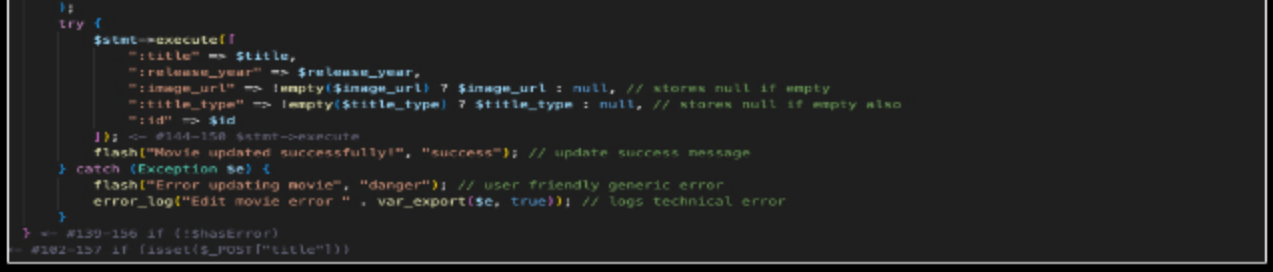
id handling

```php
// kc784 | 4/19
// uses POST data if a form was submitted or existing data for prefill
$title = se($_POST, "title", $movie["title"], false);
$release_year = se($_POST, "release_year", $movie["release_year"], false);
$image_url = se($_POST, "image_url", $movie["image_url"], false);
$title_type = se($_POST, "title_type", $movie["title_type"], false);
?>

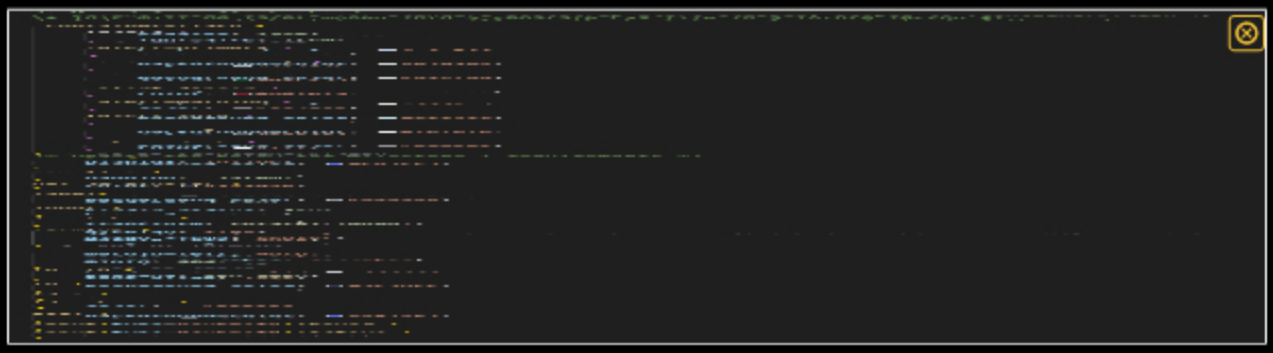
<h3>Edit Movie</h3>
<!-- require_once __DIR__ . "/partials/flash.php"; -->
<form method="POST" onsubmit="return validate();"> <!-- live js validation before submission -->
    <div>
        <label for="title">Title</label>
        <!-- live js validation runs on this -->
        <input type="text" id="title" name="title" required maxlength="255" value="<?php echo htmlspecialchars($title); ?>" />
    </div>
    <div>
        <label for="release_year">Release Year</label>
        <input type="number" id="release_year" name="release_year" min="1900" max="2030" value="<?php echo htmlspecialchars($release_year); ?>" />
    </div>
    <div>
        <label for="image_url">Image URL</label>
        <input type="url" id="image_url" name="image_url" maxlength="300" value="<?php echo htmlspecialchars($image_url); ?>" />
    </div>
    <div>
        <label for="title_type">Title Type</label>
        <!-- keep options in sync with create_movie.php -->
        <select id="title_type" name="title_type">
            <option value="">-- Select --</option>
            <option value="movie" <?php if ($title_type === 'movie') echo 'selected'; ?>>Movie</option>
            <option value="tvseries" <?php if ($title_type === 'tvseries') echo 'selected'; ?>>TV Series</option>
            <option value="short" <?php if ($title_type === 'short') echo 'selected'; ?>>Short</option>
            <option value="tvmovie" <?php if ($title_type === 'tvmovie') echo 'selected'; ?>>TV Movie</option>
            <option value="tvepisode" <?php if ($title_type === 'tvepisode') echo 'selected'; ?>>TV Episode</option>
        </select>
    </div>
    <input type="submit" value="Save Changes" /> <!-- live submit -->
</form>
```

prefill + form

```php
// kc784 | 4/19
if (!$hasError) { // only updates if all has been validated
    $stmt = $db->prepare( // prepares updates statement
        "UPDATE Movies SET title = :title, release_year = :release_year, image_url = :image_url, title_type = :title_type WHERE id = :id"
```



PHP validation + update query



styling

Saved: 4/26/2026 3:30:51 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the form is generated and submission is handled
- Briefly note the styling choices
- Note what fields are editable and why the choice was made for them (it's likely not all fields are editable, especially `id`, `created`, `modified`, etc)

Your Response:

The loading logic works through the page grabbing the movie id from the URL query. Like before, if it is less than 1 or missing, an error flash mesage is shown and a redirect is sent back to the list page. If valid, it queries the db with a prepared statement. If no record is found, "Movie not found" is displayed and redirects.

The form is generated and submissions are handled through prefilled sets of variables that check POST data then use existing movie data from the db. On submission, the validation from the create page runs with the JS validation function, PHP checks, and the HTML attributes. If everything is validated and passed, an UPDATE query modifies the existing row by id and the page refetches the movie data to solidify the changes.

The styling choice uses the same style as the create form with a centered layout, dashed white borders, the blue background, and hover effects on the buttons.

The editable fields are titles, release year, image URL, and title type just in case the user wants to change them due to incorrect spelling, date, etc. Fields like id, created, modified, api_id, etc. are

not editable because the id is the primary key, created and modified are timestamps, api_id is the external identifier.

Saved: 4/26/2026 3:30:51 PM

Task #2 (0.60 pts.) - Examples

Progress: 100%

Details:

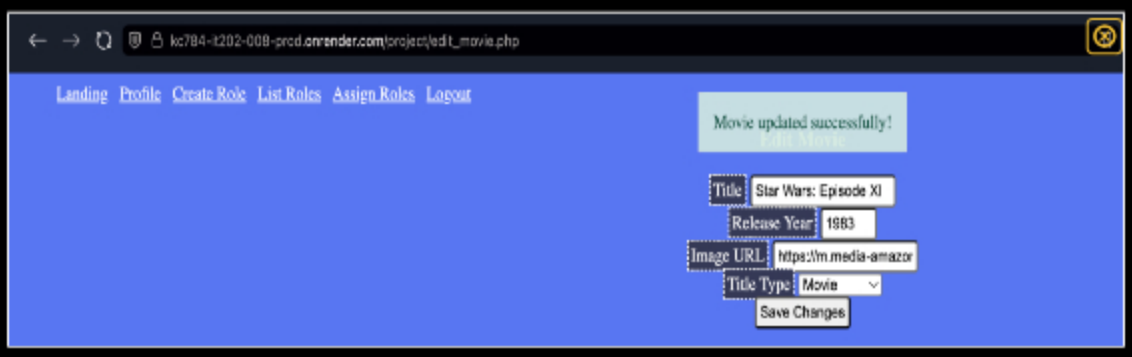
- Show a before and after of updating an entity (clearly caption)
- Successful update should have an appropriate message shown
- The updated data should be shown in the form
- Any errors should have user-friendly messages shown
- Show some examples of validation errors (html, javascript, php)



before updating



edit (changing title)

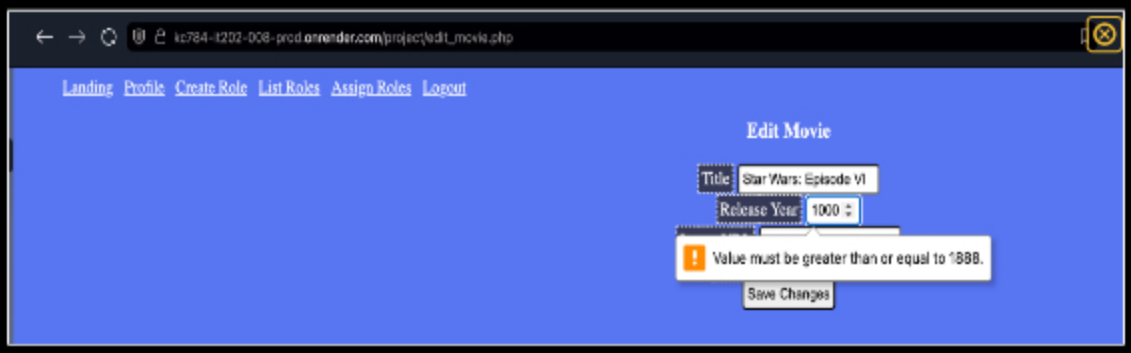


successful update

successful update



after



invalid year



PHP validation (title is required)

Saved: 4/26/2026 3:37:41 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1
https://kc784-it202-008-prod.onrender.com/project/edit_movie.php?

URL
https://kc784-it202-008-prod.

id=57

URL #2

https://github.com/kurtjcastro/kc784-it202-008-2026/pull/39

URL

https://github.com/kurtjcastr

Saved: 4/26/2026 3:38:05 PM

Section #6: (150 pts.) Delete Logic

Progress: 100%

Task #1 (0.60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles deletion of the record (and how the id retrieved for usage)
  - Include the code that handles invalid/missing ids (should redirect to the list page with an appropriate message)
- Handle any necessary roles/permissions (i.e., it's not likely that anyone can delete any entity they choose)
- Show the redirect logic that goes back to the previous page (where the route came from), if it was a list page, it should maintain the filter/sort criteria

```php
<?php
// kc784 | 4/19
// delete page to remove movie from db
// fetch movie by id, check perms, delete, redirect

require(__DIR__ . "/../../partials/nav.php");
is_logged_in(true);

$id = se($_GET, "id", -1, false); // grabs id from url

if ($id < 1) { // check if id valid
    flash("Invalid movie id", "danger");
    header("Location: list_movies.php"); // redirect to list
    die();
} // end if ($id < 1)

$db = getDB();
$stmt = $db->prepare("SELECT * FROM Movies WHERE id = :id"); // check if movie exists
$stmt->execute([":id" => $id]);
$movie = $stmt->fetch();

if (!$movie) { // movies not found
    flash("Movie not found", "danger");
    header("Location: list_movies.php");
    die();
} // end if (!$movie)
```

id retrieval

```php
<?php
// kc784 | 4/19
// delete page to remove movie from db
// fetch movie by id, check perms, delete, redirect

require(__DIR__ . "/../../partials/nav.php");
is_logged_in(true);

$id = se($_GET, "id", -1, false); // grabs id from url

if ($id < 1) { // check if id valid
    flash("Invalid movie id", "danger");
    header("Location: list_movies.php"); // redirect to list
    die();
} // end if ($id < 1)

$db = getDB();
$stmt = $db->prepare("SELECT * FROM Movies WHERE id = :id"); // check if movie exists
$stmt->execute([":id" => $id]);
$movie = $stmt->fetch();

if (!$movie) { // movies not found
    flash("Movie not found", "danger");
    header("Location: list_movies.php");
    die();
} // end if (!$movie)

// delete movie
$stmt = $db->prepare("DELETE FROM Movies WHERE id = :id");
try {
    $stmt->execute([":id" => $id]);
    flash("Movie deleted successfully", "success"); // delete success message
} catch (Exception $e) {
    flash("Error deleting movie", "danger"); // generic user friendly message
    error_log("Delete movie error: " . var_export($e, true)); // log technical error message
}
```

delete + permissions

```php
// kc784 | 4/19
// redirect to list
```

```php
$redirect = "list_movies.php";
if (isset($_GET["title"]) || isset($_GET["title_type"]) || isset($_GET["sort"]) || isset($_GET["order"]) || isset($_GET["limit"])) {
    $params = [];
    foreach(["title", "title_type", "sort", "order", "limit"] as $key) {
        if (isset($_GET[$key]) && $_GET[$key] !== "") {
            $params[$key] = $_GET[$key];
        }
    } <- #43-47 foreach(["title", "title_type", "sort", "order", "limit"] as ...
    if (!empty($params)) {
        $redirect .= "?" . http_build_query($params);
    }
} <- #41-51 if (isset($_GET["title"]) || isset($_GET["title_type"]) || is...
header("Location: $redirect");
die();
?>
```

redirect + filter preservation

```php
<a href="view_movie.php?id=<?php se($row, 'id'); ?>">View</a> |
<a href="edit_movie.php?id=<?php se($row, 'id'); ?>">Edit</a> |
<a href="delete_movie.php?id=<?php se($row, 'id'); ?>&<?php echo http_build_query($_GET); ?>" onclick="return confirm('Are you sure?')">Delete</a>
```

list page delete link

Saved: 4/26/2026 3:59:07 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the deletion logic works
- Is it a soft or hard delete?
- Note any permissions/rules applied.
- Briefly explain how the redirect maintains the previous route and any filter/sort data

Your Response:

The page grabs the movie id from the URL and checks if it's valid and queries the db to check if the movie exists. If either fail, a flash message error is shown and redirects to the list page. If the movie exists, it runs a DELETE query to remove the record and flashes a success or error message and redirects.

This delete is a hard delete as the record is permanently removed from the db with DELETE FROM Movies WHERE id = :id. Once deleted, the data cannot be recovered.

The permissions and rules are applied by having the page require the user to be logged in with is_logged_in(true) and any logged in user can delete a movie record. There are no role-based restrictions since movies are not associated with specific users for now. I will add the user-based data for movies next in M3.

The redirect preserves filters by the delete link appending the current GET params (title, type, sort, order, etc) to the delete URL with http_build_query($_GET). When the delete page is done, it reads the params from the URL and rebuilds them into a query string, appending them to the URL.

Saved: 4/26/2026 3:59:07 PM

Task #2 (0.60 pts.) - Examples

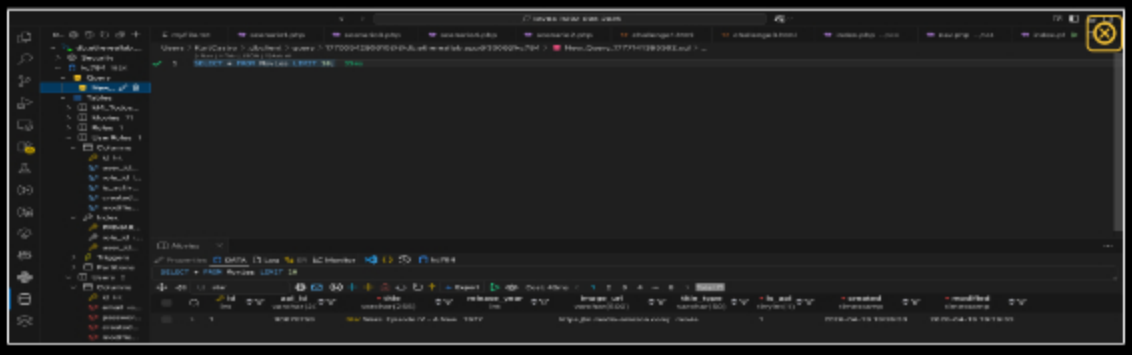
Progress: 100%

Details:

- Show a successful delete message
- Show a before and after database view of the record being deleted (clearly caption)
- Any errors should have user-friendly messages shown



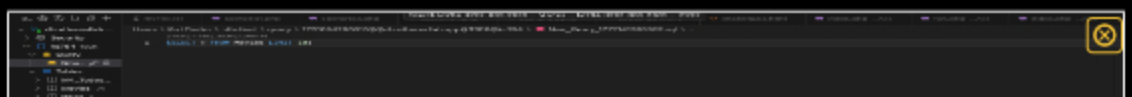
before deleting Star Wars: Episode IV - A New Hope



before deleting Star Wars: Episode IV - A New Hope (database)



after deleting Star Wars: Episode IV - A New Hope





after deleting Star Wars: Episode IV - A New Hope (database)



movie not found message

Saved: 4/26/2026 4:11:48 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page (even though it's not a full page, it's more like how logout works)
- Include pull request link for this feature

URL #1
https://github.com/kurtjcastro/kc784-it202-008-2026/pull/39

URL
https://github.com/kurtjcastr

URL #2
https://github.com/kurtjcastro/kc784-it202-008-2026/pull/40

URL
https://github.com/kurtjcastr

URL #3
https://kc784-it202-008-prod.onrender.com/project/list_movies.php

URL
https://kc784-it202-008-prod.

Saved: 4/26/2026 4:13:59 PM

Section #7: (50 pts.) Misc

Progress: 100%

Task #1 (0.17 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



commit history

Saved: 4/26/2026 4:15:28 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`) (Milestone2 into qa)

URL #1
https://github.com/kurtjcastro/kc784-it202-008-2026638

URL
https://github.com/kurtjcastro/kc

Saved: 4/26/2026 4:15:28 PM

Task #2 (0.17 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



overall time



file time

Saved: 4/26/2026 4:19:23 PM

Task #3 (0.17 pts.) - Reflection

Progress: 0%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how to fetch data and integrate data from an API into my own PHP application with API key handling on both local and deployed sites. I also learned how to better create forms with proper JS, HTML< and PHP validation, build SQL queries with proper sorting and filtering.

Saved: 4/26/2026 4:21:10 PM

Task #2 (0.33 pts.) - What was the easiest part of the

assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment were the delete and view pages because delete doesn't require a form or HTML, just taking the id and checking if it exists. If it exists, delete the record and redirect back to the list. The view page was pretty similar and just required fetching by id and displaying the fields.

Saved: 4/26/2026 4:22:15 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was getting figuring out why the API key wasn't loading properly and the path I had wasn't correctly pointing to the .env file. The fetch page was also difficult to handle because of the confusion with handling the API response and how it was structure with its nested fields to sned into my table columns as well as dealing with missing data and duplicates.

Saved: 4/26/2026 4:24:40 PM